Open SBOMs for Android Apps

Pilot report: setup, sample, six experiments (E1–E6) and limits

Pilot runs: 23 Oct 2026 (E1–E5) and 26 Oct 2026 (E6). Data: F-Droid main and archive repositories, AndroZoo. All analysis is static.

1. Summary

- The F-Droid index lists an accessibility service for 5 of 4,766 apps. The 306 sampled APKs declare 84.
- Library extraction finds 51.7 % of declared dependencies with exact artifact and version, and 76.4 % at vendor level (9 apps with source ground truth).
- All 5 showcase SBOMs and all 306 corpus SBOMs validate against CycloneDX 1.6. A broken control file fails, as it should.
- One SBOM takes 0.76 s on average to build. Covering the latest version of every F-Droid app is estimated at about 1.8 core-hours.
- Of 471 updates in 121 apps, 16 added a sensitive capability. None of the 16 shows in the F-Droid index metadata.

The pilot tests one question: can a public catalogue publish useful, valid SBOMs for Android apps from the signed APK alone? It also tests whether those SBOMs show things that current metadata does not. The sample is stratified, so the counts below describe the sample only. They are not F-Droid-wide rates.

2. Setup

Item	Detail
Main source	F-Droid main repository, `index-v2` of 18 Oct 2026 (4,766 apps), for E1–E5.
Earlier versions (E6)	F-Droid main and archive `index-v2`, both of 25 Oct 2026. AndroZoo list of 26 Oct 2026, used only to fill gaps.
Tools	androguard (manifest, accessibility config, dex strings); exodus-core 1.3.15 (Exodus Privacy tracker engine); LibScout (library profiles, JDK 8); cyclonedx-cli 0.33.1 (schema validation); own scripts for the namespace table, Maven version files and SBOM output.
Method	Static analysis only. No APK was installed or executed.
Integrity checks	Every APK was checked against the SHA-256 in its index: 306/306 sample files, 471/471 F-Droid E6 files, 24/24 AndroZoo files. For all 613 F-Droid files used in E6, the signing certificate matches the `signer` field of the index.
Hardware	One Linux machine, 8 vCPU, 15 GB RAM.

3. Sample

The sample has 306 APKs, one per app (the version in the index). It has two parts.

Stratum	How the apps were chosen	APKs
A	Index lists `BIND_ACCESSIBILITY_SERVICE`	6
B	Description mentions an accessibility service	68
C	Description mentions accessibility in another sense	58
D	Description mentions LLMs or AI	57
E	Seeded random draw from the rest of the catalogue (APKs above 80 MB skipped)	117
Total (targeted A–D: 189; random E: 117)		**306**

Strata A–D were chosen on purpose because they are likely to hold the capabilities under study. Only stratum E gives a rough idea of an ordinary app. No number in this report should be read as a rate for the whole catalogue.

4. E1: Capability scan

For each APK we read the manifest, the accessibility service config XML and the dex strings. We count sensitive capabilities and their combinations. "Read" means `canRetrieveWindowContent`. "Inject" means `canPerformGestures`.

Signal	All 306	Random E (117)	Targeted A–D (189)
Declares an accessibility service	84	3	81
Read (canRetrieveWindowContent)	41	2	39
Inject (canPerformGestures)	17	1	16
Read + inject	16	1	15
Read + inject + INTERNET	10	1	9
Overlay (SYSTEM_ALERT_WINDOW)	34	6	28
Accessibility service + overlay	22	2	20
AI endpoint string (strict host list)	33	0	33
AI SDK namespace	8	0	8
On-device AI runtime (.so: llama, ggml, onnx, whisper)	9	0	9
AI endpoint or SDK	34	0	34
Accessibility service + AI (endpoint or SDK)	6	0	6
Read + inject + AI	1	0	1
Read + inject + INTERNET + AI	0	0	0

Index comparison

- Only 5 of 4,766 index entries list `BIND_ACCESSIBILITY_SERVICE`. All 5 are in the sample. The sampled APKs declare 84 accessibility services.
- For ordinary permissions the index is accurate. It lists SYSTEM_ALERT_WINDOW for 34 sample apps, and the APK scan also finds 34.
- So the gap is specific to the service-level capability and its read/inject flags. No index field carries them.

The 10 read + inject + INTERNET apps are tools such as remote desktop, device pairing and key remapping. These are documented features. The combination describes what an app *can* do, not what it does. The one read + inject + AI app is a screen reader; its AI signal is an endpoint string, and it has no INTERNET permission. A rerun of the scripts reproduces the earlier pilot counts on all 306 apps, with no difference per package.

5. E2: Library extraction

E2 ran on 42 APKs: the 16 apps with an AI SDK or AI runtime, the 10 read + inject + INTERNET apps, and 16 random-stratum apps (seed 7).

Method	Ran?	Result
Maven version files in the APK (`META-INF/*.version`, `pom.properties`)	yes	Present in 40/42 apps (median 57 entries). The only source of exact `group:artifact@version`.
Code-namespace table (201 prefixes)	yes	Fast (median 0.16 s per APK). Fails on minified code.
Exodus tracker signatures (432)	yes	1/42 apps matches one tracker (ACRA).
exodus-core 1.3.15	yes	Same result as our signature match on 42/42 apps.
LibScout	yes, after a build fix	No public profiles exist; 6 were built. On 3 test APKs it found okhttp and retrofit in one app but missed gson, and found none of 3 declared libraries in another. Each library version needs its own profile.
LibRadar, LibPecker	no	LibRadar: Python 2 and an external feature database. LibPecker: repository no longer available.

Share of third-party classes (median over 42 apps)	Value
In known namespaces	49.2 %
Obfuscated (a/b/c-style packages)	39.6 %
Readable but unknown namespace	0.1 %
Apps with ≥ 50 % obfuscated third-party classes	20/42

Over all 306 corpus SBOMs, 151 apps have at least 50 % obfuscated third-party classes (40 of 117 in the random stratum). 269 of 306 have at least one versioned component.

Ground truth

For 9 apps we took the declared dependencies from the source code at the exact commit that F-Droid built. Gradle files and version catalogs were parsed and then checked by hand. Non-Android targets (desktop, iOS, web, tests) were removed. One more app was dropped because its dependencies are added by build helpers. One app, TypeAssist, was used earlier to extend the namespace table, so we also report the result without it.

Direct runtime dependencies	With TypeAssist (9 apps)	Without TypeAssist (8 apps)
Declared in source	203	188
Found as exact artifact + version	105 (51.7 %)	95 (50.5 %)
Found at vendor or group level	155 (76.4 %)	141 (75.0 %)
Missed	48 (23.6 %)	47 (25.0 %)

Without TypeAssist the scores drop by 1.2 and 1.4 points. The bias is real but small. A clean held-out test set would remove it fully. Main causes of misses:

- small libraries with no version file whose classes were renamed by R8;
- Compose Multiplatform artifacts, whose classes live under `androidx/` (one app accounts for 23 misses);
- libraries not yet in the namespace table.

The Maven files also reveal 42–94 transitive components per app that the source does not declare. All 4 AI SDKs declared in the ground-truth apps were detected, including in an app with 98 % obfuscated third-party classes. One MCP client SDK was missed because it is not in the pattern list yet.

6. E3: CycloneDX 1.6 SBOMs and validation

We built one SBOM per app in CycloneDX 1.6 JSON. The structure is:

- **App identity.** `metadata.component` uses `pkg:generic/<package>@<version>?download_url=…&checksum=sha256:…` plus a SHA-256 hash. `pkg:apk` is not used.
- **Components.** Libraries from Maven version files (`pkg:maven/…@version`, evidence technique `manifest-analysis`), libraries seen only by namespace (no version, `binary-analysis`), and native `.so` files.
- **Services.** AI endpoints, marked as static evidence only ("string in dex; not observed at runtime").
- **Capabilities.** Properties under a proposed `apkcap:` namespace, for example `apkcap:a11y:canRetrieveWindowContent` and `apkcap:combo:readInjectInternet`. This namespace is **not registered** in the CycloneDX property taxonomy. Each SBOM states this in a property.
- **Completeness.** `compositions` is set to `incomplete`. The dependency graph is flat because the real graph is unknown.

Showcase SBOM (version)	Why chosen	Components	AI services	Valid
`org.kde.kdeconnect_tp` 1.36.0	read + inject + INTERNET + overlay; not minified	104	0	yes
`com.typeassist.app` 4.2.0	read + AI endpoints + llama.cpp; 84 % obfuscated	79	4	yes
`io.github.patricksmill.quicknotes` 1.0.7	openai-java SDK	88	2	yes
`dev.chungjungsoo.gptmobile` 0.9.1	LiteRT-LM SDK; 98 % obfuscated	110	7	yes
`com.android.talkback` (2024-10-17 build)	read + inject + AI endpoint; no INTERNET	56	1	yes

Validation used `cyclonedx-cli 0.33.1` with `validate --input-format json --input-version v1_6 --fail-on-errors`. It reports "BOM validated successfully" for 5/5 showcase SBOMs and for 306/306 corpus SBOMs. As a negative control, we added an unknown field to one SBOM. The validator rejects it ("BOM is not valid"), as expected.

An endpoint string can come from an SDK's defaults. For example, an app using openai-java also contains the Azure OpenAI host, because the SDK supports it. A string is not proof of use.

7. E4: Cost

Per APK, 306 APKs, single run

Stage	Mean	Median	Max
APK size (MB)	14.3	6.5	84.9
Download (s, 8 parallel, F-Droid mirrors; 271 files not cached)	9.8	2.2	180.7
E1 capability scan (s)	0.44	—	—
Library extraction (s)	0.33	0.16	3.4
Full SBOM build, including scan (s)	0.76	0.35	9.0
cyclonedx-cli validation (s)	0.32	0.32	0.38
SBOM size (KB)	43	44	99

The full E1 scan of 306 APKs took 46 s of wall time with 3 workers.

Extrapolation to the latest version of all 4,766 F-Droid apps

Item	Estimate
APK data to download	86.1 GB
Download time at the measured ≈ 9.9 MB/s aggregate	≈ 2.4 h
Analysis + validation, by mean time per app	≈ 1.4 core-hours
Analysis + validation, scaled by bytes	≈ 1.8 core-hours
SBOM storage	≈ 206 MB

Assumptions: same hardware; download speed depends on the mirrors. The sample skipped very large APKs, so the bytes-based figure is the safer one. After a first full pass, only new versions need a scan, which is much cheaper.

8. E5: Comparison with Exodus Privacy

Same 42 apps as E2. Exodus Privacy answers "which known trackers and permissions does this app have?". We checked what an SBOM adds.

Measure	Count
Public Exodus report exists	39/42
Report is for the same version we scanned	28/42
Apps with at least 1 tracker in the report	1/42
exodus-core run locally gives the same trackers as our match	42/42
Read + inject apps whose Exodus page mentions accessibility	1/10
Versioned components listed in the SBOMs (not in Exodus)	2,429 (median 56 per app)
Apps with native libraries (not in Exodus)	31/42
Apps with AI endpoint, SDK or runtime signals (selected on purpose)	16/42

The SBOM adds four things: exact libraries with versions, usable for OSV and CVE matching; the accessibility read/inject configuration; AI services and runtimes; and a native code inventory with an obfuscation measure. F-Droid policy keeps trackers out of most apps, so the tracker view alone says little for this catalogue. The two approaches complement each other.

9. E6: Capability changes between versions

Question: when an app is updated through its distribution channel, does the new version gain a sensitive capability? And would someone who reads only the F-Droid index notice?

Step 1: where earlier versions can be found (306 sample apps)

Source	Count	Apps ≥ 1	Apps ≥ 2	Apps ≥ 6	Median/app
F-Droid main repo (files)	969	303	258	31	3
F-Droid archive repo (files)	4,870	188	161	129	2
F-Droid main + archive (distinct versionCodes)	5,822	306	261	159	6
AndroZoo, all markets (distinct versionCodes)	4,136	167	144	99	1
AndroZoo, market `fdroid` (files)	3,483	154	131	90	1
AndroZoo, market `play.google.com` (files)	1,476	91	72	49	0

3,360 AndroZoo files are byte-identical to a file in the current F-Droid indexes. AndroZoo adds 644 versionCodes, in 73 apps, that F-Droid no longer lists. F-Droid versionCodes include ABI splits, so there are fewer releases than versionCodes.

Step 2: version diffs

- **Apps.** 142: all 84 apps with an accessibility service and all 34 with an AI endpoint or SDK (112 distinct apps), plus 30 random others (seed 6).
- **Versions.** The sampled version and up to 5 earlier releases. One file per versionName, so ABI splits are not counted as updates. F-Droid main and archive first. AndroZoo filled gaps: 3 older F-Droid builds for 1 app, and a separate Play chain for 4 apps (21 files).
- **Lineage.** A diff is made only within one chain and one signing key. 3 updates with a signing key change were set aside. Play builds are never compared with F-Droid builds.

> **Result (F-Droid chain, same signing key).** Of **471 updates in 121 apps**, **16 updates in 15 apps** added a sensitive capability. **0 of the 16** were reflected in the F-Droid index metadata.

Added capability	Updates	Shown in index for the new version
AI endpoint string	8	0/8
Inject (canPerformGestures)	4	0/4
New accessibility service	3	0/3
AI SDK namespace	2	0/2
Read (canRetrieveWindowContent)	1	0/1
On-device AI runtime (.so), together with an AI endpoint	1	0/1
INTERNET / overlay	0 / 0	–

One update can add several capability types, so the rows sum to more than 16.

Group (same signing key)	Updates adding a capability	Apps
Accessibility apps	9 of 274	9 of 71
AI apps	9 of 111	8 of 30
Random apps	0 of 105	0 of 25

Two apps are in both the accessibility and AI groups. 6 updates *removed* a capability (overlay 4, INTERNET 1, AI endpoint 2). The added capabilities are documented features: remote control, screen reading, keyboards, launchers, AI clients. The point is not that these apps are a risk. The point is that the change cannot be seen by anyone who reads only the index.

Why the index shows 0

We compared the index with the APK for all 613 F-Droid files. The index is exact for ordinary permissions: INTERNET matches in 613/613 files, and overlay in 613/613. But 353 files declare an accessibility service, and the index lists `BIND_ACCESSIBILITY_SERVICE` for only 21 of them. The read/inject flags and AI signals have no index field at all. No anti-feature was added in any of the 16 updates. The free-text changelog mentions a related keyword in 2 of the 16, but that is prose, not metadata.

Library churn

- 134 of 471 updates (28.5 %, in 67 of 121 apps) changed at least one Maven artifact: 481 added, 196 removed, 1,704 version changes. This uses Maven version files only, so it is a lower bound.
- Where both versions carry Maven files: 129 of 446 updates.
- 15 updates added a native `.so` library. 41 added a library seen only by namespace. 1 added a known tracker signature.

In the separate Play chain (AndroZoo, 4 apps, 17 updates), 1 update added INTERNET. These Play builds are not F-Droid builds, and no index check is possible for them.

Cost. 637 version scans (capabilities, libraries, certificate) used 643 s of CPU in total: median 0.5 s, max 10.5 s per APK. Downloads: F-Droid 5.4 GB in about 11 min; 24 AndroZoo files (162 MB) in 3 min.

10. Limits

- **Sample.** Stratified on purpose. The counts do not generalise to F-Droid as a whole.
- **Static only.** A string is not proof of runtime use. Code inside native libraries, WebView JavaScript and code downloaded at runtime are not covered.
- **Namespace table.** Hand-made, 201 prefixes. It was extended once, after looking at two apps, before the 42-app run.
- **Ground truth.** 9 apps, direct dependencies only, regex parser plus manual review. R8 may have removed some declared dependencies, which inflates "missed".
- **LibScout.** Tested with 6 profiles on 3 APKs only.
- **Timings.** Single runs on one machine.
- **Namespace.** `apkcap:` is a proposal and is not registered in the CycloneDX property taxonomy.
- **E6.** Only up to 5 earlier releases that F-Droid still serves. One "update" can span several upstream releases when F-Droid skipped some. 21 of the 142 E6 apps had no earlier release on F-Droid. Library churn is a lower bound.
- **Indexes change daily.** A rerun at a later date can select other versions and give slightly different numbers.

11. Reproducibility and data

The whole pilot can be rerun with one script. E1–E5 take about 15–25 minutes, E6 about 35 minutes more, mostly downloads.

```
bash scripts/run_all.sh
```

- `scripts/`: numbered scripts 01–18 (14–18 are E6), plus shared code. `run_all.sh` downloads the indexes and APKs, and rebuilds all outputs.

- `data/`: raw outputs per experiment (`e1*` to `e6*`), the ground truth, and one scan result per E6 version.
- `sboms/`: the 5 showcase SBOMs with their validation output. `data/sboms_all/`: the 306 corpus SBOMs.
- The example SBOMs, their validation output and the negative control are attached separately (`example_sboms.zip`).

AndroZoo

Some earlier versions and the version list for E6 come from AndroZoo (K. Allix, T. F. Bissyandé, J. Klein, Y. Le Traon, "AndroZoo: Collecting Millions of Android Apps for the Research Community", MSR 2016). We thank the AndroZoo team. AndroZoo APKs are not redistributed with this pilot: only their hashes and derived scan results are kept. Access needs a personal API key, which is read from the environment and never stored.